

CT Escolinha de Vôlei

Relatório técnico do sistema — plataforma de gestão para escolinha de vôlei com painel administrativo e área gamificada do aluno.

Status atual: Fase 1 concluída — monorepo configurado, autenticação via Supabase Auth com autorização por role, CRUD completo de Alunos.

Documento gerado automaticamente a partir do estado atual do código-fonte no diretório do projeto.

1. Visão geral

O sistema atende dois perfis de usuário:

- Administrador/Professor** — controle total: cadastro de alunos, pagamentos, frequência, pontuação, ranking e relatórios de avaliação.
- Aluno/Atleta** — área individual somente leitura, para acompanhar a própria evolução (ranking, pontos, frequência, relatórios mensais).

O objetivo final vai além de um painel administrativo: a **Área do Aluno** é pensada como um produto gamificado, que motive o atleta a acompanhar sua evolução mês a mês — ranking, pontuação e relatórios de desempenho.

2. Stack técnica

Camada	Tecnologia	Papel no sistema
Frontend	React + TypeScript (Vite)	Interface web mobile-first
Roteamento	React Router	Rotas protegidas por role (admin/aluno)
Dados (cliente)	TanStack Query	Cache e chamadas à API do backend
Estado global	Context API	Sessão de autenticação (usuário/role)
Estilo	Tailwind CSS	Design system mobile-first, badges de status
Backend	Node.js + Express + TypeScript	API REST com regras de negócio
Banco de dados	Supabase (PostgreSQL gerenciado)	Persistência dos dados
Autenticação	Supabase Auth	Login, sessão, refresh de token automático
Acesso a dados (servidor)	@supabase/supabase-js (service role)	Backend ignora RLS, aplica autorização própria

3. Arquitetura de autenticação e autorização

A autenticação foi migrada de um esquema JWT/bcrypt customizado para o **Supabase Auth**, mantendo a mesma garantia de segurança exigida no briefing original: o aluno nunca consegue alterar dados administrativos, mesmo manipulando a interface.

Como funciona, passo a passo:

- O **frontend** fala diretamente com o Supabase Auth (chave anônima) para login/logout — o supabase-js guarda a sessão e renova o token automaticamente, sem lógica de refresh manual.
- O **papel do usuário** (admin ou aluno) fica armazenado em app_metadata do usuário no Supabase Auth — um campo que **só o backend** (com a chave service_role) tem permissão de escrever. O usuário não consegue alterar seu próprio papel pelo SDK client-side.
- Toda chamada do frontend ao backend Express carrega o token da sessão no cabeçalho Authorization: Bearer.
- O middleware authenticate valida esse token contra o Supabase e extrai o papel; o middleware authorize(role) bloqueia rotas administrativas para quem não é admin — **no servidor**, não apenas escondendo botões na tela.
- O backend usa a service_role key do Supabase para todas as operações de banco (ignora RLS) e para operações administrativas do Auth (criar/remover usuário, definir role). Essa chave nunca é exposta ao frontend.

Como camada extra de defesa, todas as tabelas têm Row Level Security (RLS) habilitado e sem políticas — mesmo que a chave anônima vazasse, nenhuma tabela seria acessível por ela.

4. Modelo de dados

O schema completo do domínio já está modelado em `backend/supabase/schema.sql`, mesmo para entidades que só entram em fases futuras — isso evita migrations retroativas.

Entidade	Descrição	Fase
usuarios	Espelha <code>auth.users</code> : nome, email, role (admin/aluno)	1 — em uso
alunos	Dados do atleta: nascimento, telefone, turma, status, foto	1 — em uso
turmas	Nome e horários da turma	2
treinos	Data, horário, local, tipo, turma	2
frequencias	Presença por treino (presente/falta/falta justificada)	2
mensalidades	Mês de referência, valor, vencimento, status de pagamento	2
pontuacoes	Histórico de pontos com motivo e data	3
relatorios	Avaliação mensal: técnico/físico/tático/mental + observações	3

O **ranking** é intencionalmente calculado a partir da soma de `pontuacoes` por período/turma, e não persistido como tabela própria — evita dados redundantes e inconsistência entre pontuação e ranking.

5. Regra de negócio central (LGPD)

Dados financeiros e pessoais estão envolvidos, então a autorização por role é aplicada **no backend**, nunca só no frontend:

- Todas as rotas de escrita (criar, editar, ativar/desativar, remover aluno) exigem `authorize("admin")`.
- O aluno só acessa `GET /alunos/me` — leitura do próprio perfil, sem nenhuma rota de escrita disponível para o papel aluno.
- Senhas nunca passam pelo backend em texto plano além do momento da criação — o hashing e a validação de login são responsabilidade do Supabase Auth.

6. O que já está implementado (Fase 1)

- Monorepo `backend/ + frontend/` configurado e validado (typecheck e build de produção rodados com sucesso em ambos).
- Schema SQL completo para o Supabase, com RLS habilitado.
- Login (admin e aluno) com sessão persistente e renovação automática de token.
- Middleware de autenticação e autorização por role no Express.
- CRUD completo de Alunos pelo admin: criar (com criação do usuário vinculado no Supabase Auth), listar com filtro por status, editar, ativar/desativar, remover, ver perfil individual.
- Área do aluno protegida por role, com visualização do próprio perfil.
- Script `seed:admin` para criar o usuário administrador inicial.

7. Próximas fases

Fase	Conteúdo
2 — Operação diária	Turmas, Treinos, Frequência (presença + %), Financeiro (mensalidades)
3 — Engajamento	Pontuação com histórico, ranking mensal calculado, relatório individual mensal
4 — Área do aluno e dashboard	Dashboard do admin (contadores), home gamificada do aluno, histórico mensal comparativo
5 — Polimento	Responsividade fina, estados de loading/erro, validações de formulário

8. Como rodar o projeto

Passo a passo resumido (detalhes completos em README.md):

- Criar um projeto no Supabase e rodar backend/supabase/schema.sql no SQL Editor.
- `cd backend && npm install`, configurar `.env` com `SUPABASE_URL` e `SUPABASE_SERVICE_ROLE_KEY`, rodar `npm run seed:admin` e depois `npm run dev`.
- `cd frontend && npm install`, configurar `.env` com `VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY`, rodar `npm run dev`.
- Acessar `http://localhost:5173` e entrar com `admin@ctescolinha.com / admin123`.